Top 35 Frequently asked Angular Interview Questions

1. What is Angular?

Angular is a JavaScript-based framework developed by Google. Angular allows developers to build applications for browsers, mobile, and desktop using web technologies like HTML, CSS, and JavaScript.

Angular2+ is different from Angular1.x and it is completely re-written from scratch using Typescript. At the time of writing this, book the latest version of Angular was 13.

2. What is Component in Angular?

An Angular component mainly contains a template, class, and metadata. It is used to represent the data visually. A component can be thought as a web page. An Angular component is exported as a custom HTML tag like as:
<my-app></my-app>.

When we create new create an Angular app using CLI, it will create default App component as the entry point of application. To declare a class as a Component, we can use the Decorators and decorator to declare a component looks like the below example.

```
import{Component}from'@angular/core';

@Component({
selector:'my-app',
template: `<h1>{{ message }}</h1>`,
styleUrls:['./app.component.css']
})
exportclassAppComponent{
message = "Hello From Angular";
}
```

For creating a component, we can use decorator @component like the above example. If we are using Angular CLI than new component will be created using the below ng command.

ng generate component componentName

3. What is Data Binding in Angular?

Data binding is one of the important core concepts which is used to do the communication between the component and DOM.

In other words, we can say that the Data binding is a way to add/push elements into HTML Dom or pop the different elements based on the instructions given by the Component.

There are mainly three types of data binding supported in order to achieve the data bindings in Angular.

- One-way Binding (Interpolation, Attribute, Property, Class, Style)
- Event Binding
- Two-way Binding

Using these different binding mechanisms, we can easily communicate the component with the DOM element and perform various operations.

4. What is Directive in Angular?

The directive in Angular basically a class with the (@) decorator and duty of the directive is to modify the JavaScript classes, in other words, we can say that it provides metadata to the class.

Generally, @Decorator changes the DOM whenever any kind of actions happen and it also appears within the element tag.

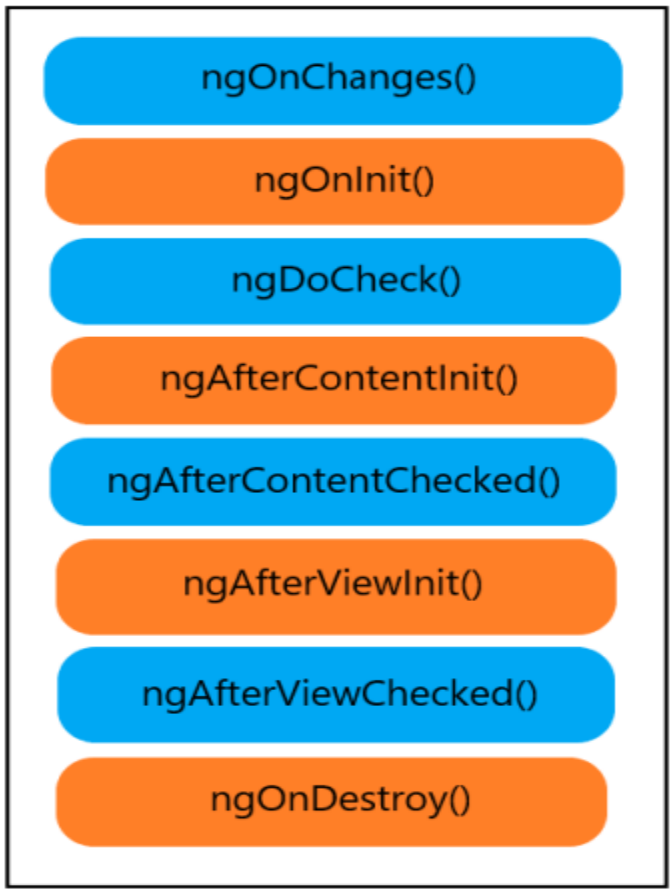
The component is also a directive with the template attached to it, and it has template-oriented features. The directive in Angular can be classified as follows.

1. Component Directive
2. Structural Directive
3. Attribute Directive

5. What are various Angular Lifecycle hooks?

Angular has its own Component lifecycle, and every lifecycle event will occur based on the data-bound changes.

Right from the initialization of a component to the end of the application, events of the lifecycle are processed based on the current component status.



ngOnChanges()

This is the first method to be called when any input-bound property will be set and always respond before ngOnInit() whenever the value of the input-bound properties are changed.

ngOnInit()

It will initialize the component or a directive and set the value of component's different input properties. And one important point is that this method called once after ngOnChanges ().

ngDoCheck()

This method normally triggered when any change detection happens after two methods ngOnInit() and ngOnChanges().

ngAfterContentInit()

This method will be triggered whenever component's content will be initialized and called once after the first run through of initializing content.

ngAfterContentChecked()

Called after every ngDoCheck() and respond after the content will be projected into our component.

ngAfterViewInit()

Called when view and child views are initialized and called once after view initialization.

ngAfterViewChecked()

Called after all of the content is initialized, it does not matter either there are changes or not but still this method will be invoked.

ngOnDestroy()

Called just before destroys of a component or directive and we need to unsubscribe any pending subscription which can harm us as extreme memory leaks.

6. Methods to Share Data between Angular Components

While developing an application, there is a possibility to share data between the components.

And sharing data between the components is our day-to-day kind of stuff where communication between components is a must either parent to child, child to the parent or communicate between un-related components.

To enable data sharing between components, different ways are possible which are listed below.

- Sharing data using @Input()
- Sharing data using @Output and EventEmitter
- Sharing data using @ViewChild
- Using Services

7. What is Module in Angular?

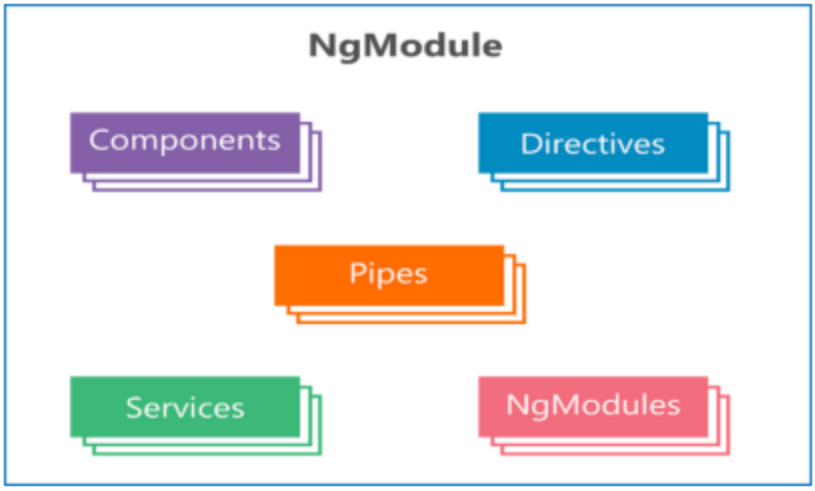
The module in Angular is a very essential part of the application, which is used to organize our angular app into a distributive manner.

Or we can say that Modules plays an important role to structure our Angular application.

Each Angular application must have one parent module which is called app module and it contains the list of different items which are listed below.

- Components
- Service providers
- Custom modules
- Routing modules
- Pipes

Module file group all of them into a single file; the same module will be useful for bootstrapping the application.



8. What different types of Modules are supported by Angular?

Modules in Angular are categorized into 5 types which are listed below.

- Routing module
- Service module
- Features module
- Shared module

9. What is Entry Component?

The bootstrapped component is an entry component which loads the DOM during the bootstrapping process. In other words, we can say that entry components are the one, which we are not referencing by type, thus it will be included during the bootstrapping mechanism.

There are mainly two types of entry component declarations can be possible.

1. Using bootstrapped root component

```
import{ NgModule } from'@angular/core'; import{ CommonModule } from'@angular/common';
import{ MymoduleRoutingModule } from'./mymodule-routing.module';

@NgModule({ declarations: [], imports: [
CommonModule, MymoduleRoutingModule
]
})
Export classMymoduleModule{ }
```

We can specify our component into bootstrap metadata in an app.module file like this.

2. Using route definition

```
Const myRoutes: Routes = [
{
path:'',
component: DashboardComponent
}
];
```

Another way to define the entry component is to use it along with the routing configuration like this.

10. What is Pipe in Angular?

During the web application development, we need to work with data by getting it from the database or another platform, at that we get the data into a specific format and we need to transform it to a suitable format, and for this purpose, we can use pipes in Angular.

The pipe is a decorator and it is used to transform the value within the template.

It is just a simple class with @Pipe decorator with the class definition and it should implement the PipeTransform interface which accepts the input value and it will return the transformed value.
To create a new pipe, we can use the below command.

11. What are built-In Pipes in Angular?

There are different types of in-built pipes available in Angular, which are listed below.

- Date pipes
- Currency pipes
- Uppercase pipes
- Lowercase pipes
- Titlecase pipes
- Async pipes
- Slice pipes
- Number pipes
- Json pipes

And many more types of pipe are available to use for data transformation.

12. What is Routing in Angular?

Routing in Angular allows the user to navigate to the different page from the current page. When we try to enter URL in the browser at a time it navigates to a specific page and performs appropriate functions implemented into the pages.

Routing is one of the crucial tasks for any application, because the application may have a large number of pages to work with.

13. How to use Routing in Angular?

Angular has its own library package @angular/router it means that routing is not a part of core framework @angular/core package.

In order to use Routing in our Angular application, we need to import RouterModule like this into our root module file.

```
import{ Routes, RouterModule } from'@angular/router';
```

14. What is the Route in Angular?

The route is the Array of different route configuration and contains the various properties which are listed below.

Property	Description
Path	String value which represents route, matcher
pathMatch	Matches the path against the path strategy
Component	The component name which is used to indicate the component type
Matcher	To configure custom path matching
Children	Children are the array of different child routes

loadChildren	Load children are used to loading the child routes as lazily loaded routes

These are the primary properties and also other properties are supported like data, canActivate, resolve, canLoad, canActivateChild, canDeactivates.

And by using these different properties, we can perform many operations like loading the child route, matching the different route paths; specify the route path for specific component etc.

15. What are the Building blocks of an Angular Form?

While working with the Forms in Angular, we are using different form controls to get the user inputs, based on that we have some basic building blocks which are listed below.

- **FormGroup**

It has the collection of all FormControls used specifically for a single form.

- **FormControl**

FormControlis used to manage the form controls value and also maintain the status of the validation.

- **FormArray**

Manages the status and value for Array of the form controls

- **ControlValueAccessor**

Acts as a mediator between FormControl and our different native DOM elements.

16. What are the differences between Reactive form and Template-driven form in Angular?

The differences between Reactive Form and Template-driven Form are given below:

Reactive Forms	Template-driven Forms
Supports dynamic form and structure	Supports static form approach
Form structure will be implemented in Typescript or else you can say in the code itself	Form structure will be implemented in the HTML template
Form values passed to the code using forms value property	It allows one-way and two-way data binding to pass the forms value
Behavior is Synchronous	Behavior is Asynchronous

17. What is FormControl in Angular and how to use it?

To create a reactive form, we use FormGroup, FormControl, and FormArray with the ReactiveFormsModule.

FormControl is generally used to track the form values and also helps to validate the different input elements. To use FormControl, we need to import it into the module file like this.

```
Import { FormsModule, ReactiveFormsModule } from'@angular/forms';
```

Also, we need to add the same inside the imports array.

```
imports: [ BrowserModule, FormsModule, ReactiveFormsModule],
```

To use FormControl with the template, you can see the simple example like this.

```
<table>
<tr>
<td>Enter Your Name :</td>
<td><input type="text" [formControl]="fullName"/></td>
</tr>
</table>
```

And inside the component file, we need to import FormControl and create new instance as described below.

```
import{ Component } from '@angular/core';
import{ FormControl, Validators } from '@angular/forms';
@Component({ selector:'my-app',
templateUrl:'./app.component.html', styleUrls: [ './app.component.css' ]
})
Export classAppComponent  {
fullName = new FormControl('', Validators.required);
}
```

18. What is Dependency Injection in Angular?

You may have heard this term called Dependency Injection so many times, but many of them don't know the actual importance and meaning about that.

Dependency Injection is an application design pattern which is used to achieve efficiency and modularity by creating objects which depend on another different object.

In AngujarJs 1.x, we were using string tokens to get different dependencies, but in the Angular newer version, we can use type definition to assign providers like below example.

```
constructor(private service: MyDataService) { }
```

Here in this constructor, we are going to inject MyDataService, so a whenever instance of the component will be created at that time it determines that which service and other dependencies needed for a component by just looking the parameters of the constructor.

In short, it shows that current component completely depends on MyDataService, and after instance of the component will be created than requested service will be resolved and returned, and at last constructor with the service argument will be called.

19. What are many ways to implement Dependency Injection in Angular?

In order to implement DI, we should have one provider of service which we are going to use, it can be anything like register the provider into modules or component.

Below are the ways to implement DI in Angular.

- Using @Injectable() in the service
- Using @NgModules() in the module
- Using Providers in the component

20. What are Observables in Angular?

Observables is a way to populate the data from the different external resources asynchronously.

Observable replaces the promises in most of the cases like Http, and the major difference between the Promises and Observable is that Observable can be used to observe the stream is different events.

Observable is declarative it means that when we define a function for publishing but it won't be executed until any consumer subscribes it.

Due to asynchronous behavior of Observable, it is used extensively along with Angular because it is faster than Promise and also can be canceled at any time.

21. What is Angular Service?

While developing app we need to change or manipulate data constantly, and in order to work with data, we need to use service, which is used to call the API via HTTP protocol.

Angular Service is a simple function, which allows us to create properties and method to organize the code.

To generate services, we can use below command which generates service file along with default service function structure.
ng generate service mydataservice

22. Why we should use Angular Services?

Services in Angular are the primary concern for getting or manipulate data from the server. The primary use of the services is to invoke the function from the different files like Component, Directives and so on.

Normally services can be placed separately in order to achieve reusability as well as it will be easy to distribute the Angular application into small chunks. Using dependency injection, we can inject the service into the Component's constructor and can use different functions of service into the whole component.

One of the main advantages is Code Reusability because when we write any function inside the service file then it can be used by the different component so that we don't need to write same methods into multiple files.

23. What is HTTP Client in Angular?

While working with any web-application, we always need the data to work with and for that, we need to communicate with the database which resides on a server.

In order to communicate from the browser, it supports two types of API.

- Fetch() API
- XMLHttpRequest

Same way in Angular, we have HTTP API which is based on XMLHttpRequest which is called HttpClient to make a request and response.

Before Angular 5, HTTP Module was used but after that, it was replaced by the HttpClientModule. To use HttpClientModule in our Angular app, we need to import it like this.

```
import{ HttpClientModule } from '@angular/common/http';
```

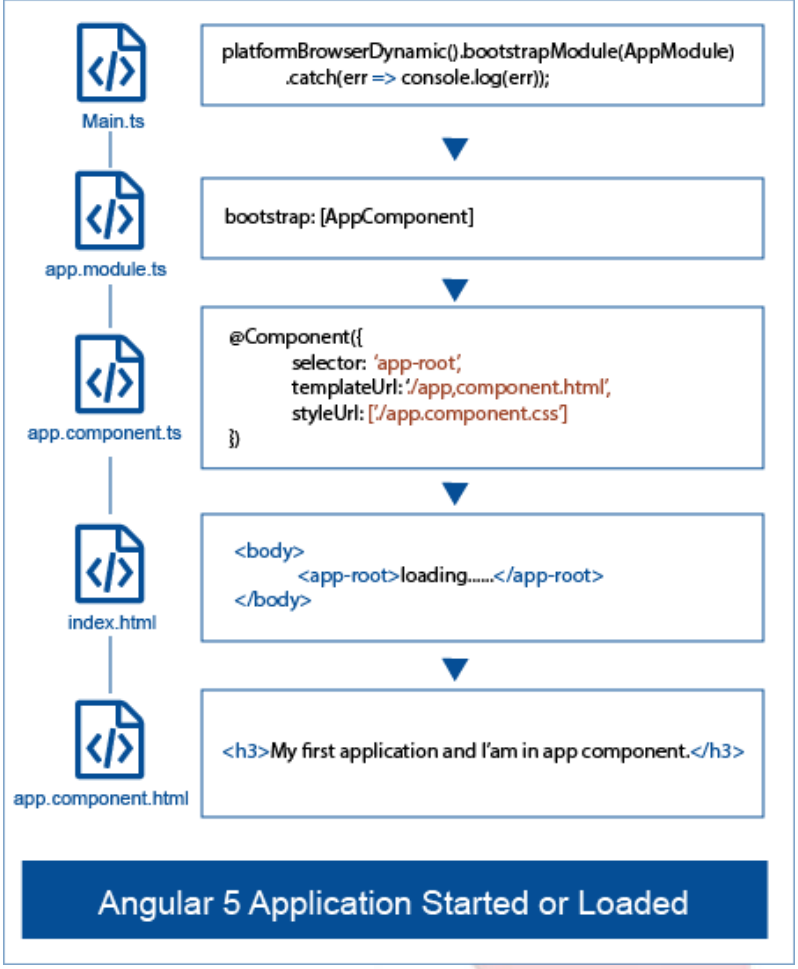
24. What is Interceptor?

The inception of HTTP request and response is one of the major parts of HTTP package and using inceptor we can transform the HTTP request from our angular application to the communication server, and the same way we can also transform the response coming from the server.

If we don't use the interceptors then we need to implement the tasks logic explicitly whenever we send the request to the server.

25. How does an Angular application get start?

An Angular application gets loaded or started by the below ways.



26. What is webpack?

Webpack is a tool used by an Angular CLI to create bundles for JavaScript and stylesheets. Webpack injects these bundles into index.html file at runtime.

27. What is the difference between AOT and JIT?

In Angular we have AOT (Ahead of Time) and JIT (Just In Time) Compilation for compile the application. There are few differences between JIT and AOT:

- JIT stands for Just In Time so by the name you can easily understand its compile the while AOT stands for Ahead of Time so according to the name
- According to compilation, JIT loads the application slowly while application more quickly as compared to that of JIT.
- In case of JIT, Template Binding errors will show at the time of showing the application while in case of AOT Template binding errors will show at the building time.
- In case of because in case of AOT the bundle

28. What is the purpose of main.ts file?

The main.ts file is the main file which is the start point of our application. As you have read before about the main method the same concepts are here in the Angular application. It's the file for the bootstrapping the application by the main module as .bootstrapModule (AppModule). It means according to main.ts file, Angular loads this module first.

29. What is ViewEncapsulation?

ViewEncapsulation decides whether the styles defined in a component can affect the entire application or not. There are three ways to do this in Angular:

Emulated: When we will set the ViewEncapsulation.Emulated with the @Component decorator, Angular will not create a Shadow DOM for the component and style will be scoped to the component.One thing need to know necessary it's the default value for encapsulation.

Native: When we will set the ViewEncapsulation.Native with the @Component decorator,Angular will create Shadow DOM for the component and style will create Shadow DOM from the component so style will be show also for child component.

None: When we will set the ViewEncapsulation.None with the @Component decorator, the style will show to the DOM's head section and is not scoped to the component. There is no Shadow DOM for the component and the component style can affect all nodes in the DOM.

30. How many types of HTTP Requests are supported?

There are total 8 types of methods are supported which are listed below.

- GET Request
- Post Request
- Put Request
- Patch Request
- Delete Request
- Head Request
- Jsonp Request
- Options Request

31. What are different types of methods in RouterModule?

There are mainly two static methods are available which are listed below.

1. forRoot(): It performs initial navigation and contains the list of configured router providers and directives. You can find the simple example using forRoot below.
RouterModule.forRoot([
{ path:'', redirectTo:'dashboard', pathMatch:'full' },
{ path:'dashboard', component:DashboardComponent },
{ path:'about', component:AboutComponent },
{ path:'contact', component:ContactComponent }
])
2. forChild(): forChild is used to create the module with all of the router directive and provider registered routes.

32. What is HTTPClient in Angular?

While working with any web-application, we always need the data to work with and for that, we need to communicate with the database which resides on a server.
In order to communicate from the browser, it supports two types of API.

i. Fetch() API
ii. XMLHttpRequest

Same way in Angular, we have HTTP API which is based on XMLHttpRequest which is called HttpClient to make a request and response.
Before Angular 5, HTTP Module was used but after that, it was replaced by the HttpClientModule.

To use HttpClientModule in our Angular app, we need to import it like this. import{ HttpClientModule } from '@angular/common/http';

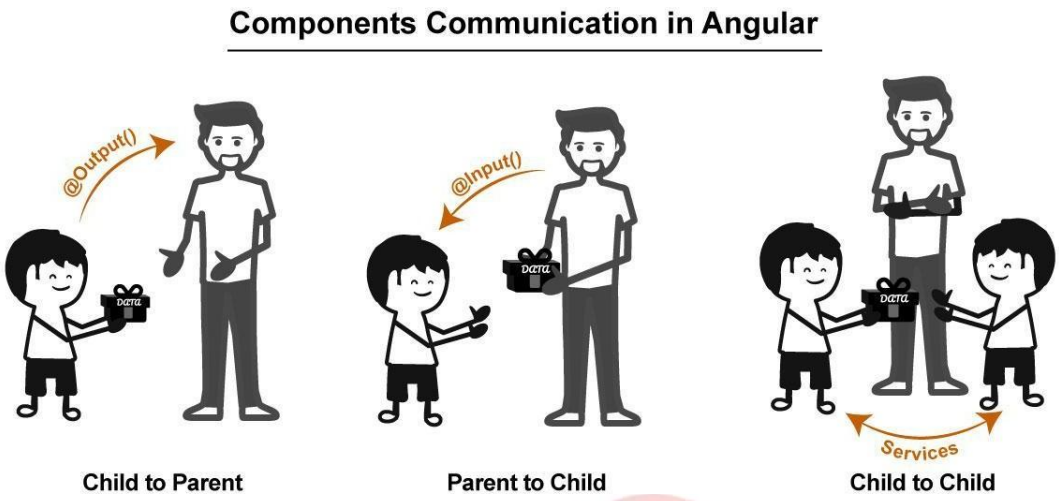
33. What is next object in Interceptor?

We have seen the simple example about HttpInterceptor in which we have used next object, which is used to represent next interceptor in a queue to be transformed.
Next object will be used whenever we have the chain of the interceptor and by using next object, we will get the response as Observable.
By using below the line, we can handle the next interceptor in a chain of the interceptor.
next.handle()

34. How we can handle the error in Angular?

We can handle the error using try catch in the Angular application also we can handle the error using error section when subscribe the observables.

35. How can we pass data from component to component?



We can pass the data from component either parent and child relationships or not. If component are separate and we want to pass data between component so we can use services with get and set properties.set for setting the values which you want and get for getting the values which added by one component .